Intro to Pandas

আমরা যখন ডেটা সায়েন্স বা মেশিন লার্নিং নিয়ে কাজ করি, তখন আমাদের সবচেয়ে গুরুত্বপূর্ণ জিনিস হচ্ছে — ডেটা **(Data)**।

এই ডেটা কখনও সঠিক ফরম্যাটে থাকে না — কখনও মিসিং ভ্যালু, কখনও ভুল কলাম, কখনও অপ্রয়োজনীয় ইনফরমেশন থাকে।

এই অবস্থায় ডেটাকে সাজানো, পরিষ্কার করা, এবং বিশ্লেষণের উপযুক্ত করে তোলা — এটিই হলো ডেটা প্রিপ্রসেসিং **(Data Preprocessing)**।

আর এই কাজটি করতে যে লাইব্রেরি আমাদের সবচেয়ে বেশি সাহায্য করে, সেটি হলো **Pandas**।

Pandas কী?

Pandas হলো Python এর একটি শক্তিশালী ওপেন-সোর্স লাইব্রেরি যা মূলত ডেটা বিশ্লেষণ **(Data Analysis)** এবং ডেটা ম্যানিপুলেশন **(Data Manipulation)** এর জন্য ব্যবহৃত হয়।

এর নাম এসেছে "**Panel Data**" থেকে — যার মানে হলো বহু-মাত্রিক (multidimensional) ডেটা সেট।

কেন Pandas দরকার?

১. ডেটা ক্লিনিং **(Data Cleaning)**

Pandas দিয়ে খুব সহজেই আমরা —

- মিসিং ভ্যালু (Missing Values) হ্যান্ডেল করতে পারি
- ডুপ্লিকেট ডেটা রিমুভ করতে পারি
- ডেটার ফরম্যাট ঠিক করতে পারি

২. ডেটা প্রিপ্রসেসিং **(Data Preprocessing)**

মেশিন লার্নিং মডেল ট্রেন করার আগে ডেটাকে প্রস্তুত করতে হয়।

Pandas দিয়ে আমরা সহজেই:

- কলাম যোগ/বাদ দিতে পারি
- ডেটা স্কেল বা নর্মালাইজ করতে পারি
- কেটেগোরিকাল ডেটাকে নিউমেরিক্যাল এ রূপান্তর করতে পারি (যেমন get_dummies() ব্যবহার করে)

৩. ডেটা অ্যানালাইসিস **(Data Analysis)**

Pandas দিয়ে আমরা ডেটার বিভিন্ন দিক বিশ্লেষণ করতে পারি:

- গড় (mean), মিডিয়ান (median), সর্বাধিক মান (max/min)
- গ্রুপ-বাই (groupby) করে বিভিন্ন সাবসেট বিশ্লেষণ
- ডেটার উপর ফিল্টার ও কন্ডিশন প্রয়োগ

৪. ডেটা ভিজুয়ালাইজেশন **(Data Visualization)**

যদিও Pandas মূলত ডেটা হ্যান্ডলিং লাইব্রেরি, তবুও এটি **Matplotlib** বা **Seaborn** এর সাথে যুক্ত হয়ে সুন্দর গ্রাফ তৈরি করতে পারে।

৫. মেশিন লার্নিং এ **Pandas** এর ভূমিকা

Pandas মেশিন লার্নিং এর প্রথম ধাপেই সবচেয়ে গুরুত্বপূর্ণ:

- ডেটা লোড করা (pd.read_csv(), pd.read_excel())
- ডেটা ক্লিন করা (মিসিং ভ্যালু, আউটলায়ার হ্যান্ডল)

- ফিচার তৈরি করা (Feature Engineering)
- মডেলে ইনপুট দেওয়ার আগে ডেটাকে প্রস্তুত করা

বলা যায়, **"No Clean Data, No Good Model"** — আর সেই ক্লিনিং এর মূল হাতিয়ার হলো **Pandas**।

Pandas DataFrame and Series

ভূমিকা

Pandas মূলত দুটি গুরুত্বপূর্ণ ডেটা স্ট্রাকচার ব্যবহার করে কাজ করে —

Series

DataFrame

এই দুইটি ডেটা স্ট্রাকচার একে অপরের সাথে সম্পর্কিত এবং প্রায় সব ধরনের ডেটা বিশ্লেষণ বা প্রিপ্রসেসিং এ ব্যবহার হয়। চলো আগে একটু বুঝে নিই এরা কী।

1. Pandas Series কী?

Series হচ্ছে একটি এক-মাত্রিক (1D) ডেটা স্ট্রাকচার, যেখানে একটি নির্দিষ্ট ইনডেক্স সহ ডেটার লিস্ট রাখা হয়।
তুমি একে Python-এর **list** বা **array** এর মতো ভাবতে পারো, তবে এর একটি **label (index)** থাকে যা ডেটা আইটেমগুলোকে সনাক্ত করে।

উদাহরণ:

```
1
2import pandas as pd
3
4numbers = pd.Series([10, 20, 30, 40, 50])
5print(numbers)
6print(type(numbers))
```

আউটপুট:

```
1
20    10
31    20
42    30
53    40


64    50
7dtype: int64
8<class 'pandas.core.series.Series'>
```

ব্যাখ্যা:

> বামপাশে 0, 1, 2, 3, 4 হলো **index**
> ডানপাশে 10, 20, 30, 40, 50 হলো **values**
> Pandas স্বয়ংক্রিয়ভাবে index তৈরি করে, তবে চাইলে আমরা নিজের index সেট করতে পারি।

```
1
2marks = pd.Series([85, 90, 78], index=['Alif', 'Fatima', 'Imran'])
3print(marks)
```

আউটপুট:

```
1
2Alif      85
3Fatima    90
4Imran     78
5dtype: int64
```

2. Pandas DataFrame কী?

DataFrame হলো একটি দুই-মাত্রিক (2D) ডেটা স্ট্রাকচার।
এক কথায় — এটি হচ্ছে টেবিল ফরম্যাটে ডেটা রাখার ব্যবস্থা, যেখানে প্রতিটি কলাম আসলে একটি **Series**।
অর্থাৎ, একটি DataFrame হলো **Series-**এর সংগ্রহ **(collection of Series)**।
তুমি এটি Excel sheet বা SQL table-এর মতো কল্পনা করতে পারো।

উদাহরণ: CSV ফাইল থেকে DataFrame তৈরি

```
1
2import pandas as pd
3
4df = pd.read_csv('student_data.csv')
5print(df)
6print(type(df))
```

আউটপুট (সংক্ষেপে):

```
1
2   StudentID          FullName  ...  Instructor    Location
30     PH1001       Alif Rahman  ...   Mr. Karim       Dhaka
41     PH1002     Fatima Akhter  ...   Ms. Salma  Chattogram
```

```
52     PH1003     Imran Hossain  ...   Mr. Karim       Dhaka
6...
719    PH1020       Nasir Khan   ...   Ms. Salma       Dhaka
8
9[20 rows x 9 columns]
10<class 'pandas.core.frame.DataFrame'>
```

ব্যাখ্যা:

> df একটি DataFrame, অর্থাৎ দুই-মাত্রিক টেবিল।
> প্রতিটি কলাম হলো একটি Series।
> এখানে মোট ২০টি row এবং ৯টি column আছে।
> type(df) দিলে দেখা যায় এটি pandas.core.frame.DataFrame।

3. DataFrame থেকে Series অ্যাক্সেস করা

যেহেতু প্রতিটি কলামই এক একটি Series, তাই আমরা সহজেই সেগুলো আলাদা করে ব্যবহার করতে পারি।

```
1
2names = df['FullName']
3print(names)
4print(type(names))
```

আউটপুট:

```
1
20          Alif Rahman
31        Fatima Akhter
42        Imran Hossain
5...
619          Nasir Khan
7Name: FullName, dtype: object
8<class 'pandas.core.series.Series'>
```

ব্যাখ্যা:

এখানে df['FullName'] একটি Series।

অর্থাৎ, DataFrame এর প্রতিটি কলাম আলাদা করে এক-মাত্রিক Series হিসেবে ব্যবহৃত হয়।

4. DataFrame তৈরি করা (ম্যানুয়ালি)

চাইলে DataFrame নিজে তৈরি করাও যায়, dictionary ব্যবহার করে।

```
data = {
    'StudentID': ['PH1001', 'PH1002', 'PH1003'],
    'Name': ['Alif', 'Fatima', 'Imran'],
    'Marks': [85, 90, 78]
}

df2 = pd.DataFrame(data)
print(df2)
```

আউটপুট:

```
  StudentID    Name  Marks
0    PH1001    Alif     85
1    PH1002  Fatima     90
2    PH1003   Imran     78
```

5. DataFrame এর কিছু সাধারণ অ্যাট্রিবিউট

df.shape	রো এবং কলামের সংখ্যা দেখায়
df.columns	সব কলামের নাম দেখায়
df.head()	প্রথম কয়েকটি রো দেখায়
df.tail()	শেষ কয়েকটি রো দেখায়
df.info()	ডেটা টাইপ এবং নাল ভ্যালু সম্পর্কে তথ্য দেয়
df.describe()	সংখ্যাগত কলামগুলির পরিসংখ্যান দেখায়

উদাহরণ:

```
print(df.shape)
print(df.columns)
print(df.head())
```

Working with Different file types in Pandas

ভূমিকা

ডেটা সায়েন্স বা মেশিন লার্নিং প্রজেক্টে ডেটা বিভিন্ন ফরম্যাটে সংরক্ষণ করা হয়। যেমন:

- .csv (Comma-Separated Values)
- .xlsx (Excel)
- .parquet (Compressed Binary Format)
- .json (JavaScript Object Notation)

এই সব ফাইল থেকেই আমরা Pandas DataFrame তৈরি করতে পারি এবং সহজেই বিশ্লেষণ বা প্রক্রিয়াজাত করতে পারি।
এই মডিউলে আমরা দেখব কীভাবে Pandas দিয়ে বিভিন্ন ফাইল টাইপ লোড করা যায়।

1. CSV ফাইল লোড করা

CSV (Comma-Separated Values) হলো সবচেয়ে জনপ্রিয় এবং সাধারণ ডেটা ফরম্যাট।
Pandas দিয়ে CSV ফাইল পড়া যায় pd.read_csv() ফাংশন ব্যবহার করে।

```
1
2import pandas as pd
3
4# csv file
5csv_data = pd.read_csv('student_data.csv')
6
7print(csv_data)
8print(type(csv_data))
```

আউটপুট (সংক্ষেপে):

```
1
2   StudentID          FullName  ...  Instructor    Location
30     PH1001       Alif Rahman  ...   Mr. Karim       Dhaka
41     PH1002     Fatima Akhter  ...   Ms. Salma  Chattogram
52     PH1003     Imran Hossain  ...   Mr. Karim       Dhaka
```

```
6...

719    PH1020        Nasir Khan  ...   Ms. Salma       Dhaka

8

9[20 rows x 9 columns]

10<class 'pandas.core.frame.DataFrame'>
```

ব্যাখ্যা:

- pd.read_csv() ফাংশন CSV ফাইল থেকে ডেটা পড়ে এনে DataFrame তৈরি করে।
- ফাইলের প্রতিটি কলাম আলাদা Series হিসেবে সংরক্ষণ হয়।
- type(csv_data) দিলে বোঝা যায় এটি একটি DataFrame।

2. Excel ফাইল লোড করা

Excel ফাইল সাধারণত .xlsx এক্সটেনশনে থাকে।
এটি পড়তে pd.read_excel() ব্যবহার করা হয়।

```
1

2# excel file

3excel_file = pd.read_excel('phitron_student_marks.xlsx')

4

5print(excel_file)

6print(type(excel_file))
```

আউটপুট:

```
1

2   StudentID          FullName  ...  Instructor    Location

30     PH1001       Alif Rahman  ...   Mr. Karim       Dhaka

41     PH1002     Fatima Akhter  ...   Ms. Salma  Chattogram

5...

619    PH1020        Nasir Khan  ...   Ms. Salma       Dhaka

7

8[20 rows x 9 columns]
```

```
9<class 'pandas.core.frame.DataFrame'>
```

ব্যাখ্যা:

Excel ফাইল পড়ার জন্য সিস্টেমে openpyxl বা xlrd লাইব্রেরি ইন্সটল থাকতে হয়।

pd.read_excel() স্বয়ংক্রিয়ভাবে প্রথম শীট পড়ে নেয়।

যদি ফাইলে একাধিক শীট থাকে, তবে sheet_name প্যারামিটার ব্যবহার করা যায়।

উদাহরণ:

```
1

2pd.read_excel('file.xlsx', sheet_name='Sheet2')
```

3. Parquet ফাইল লোড করা

Parquet হলো একটি binary, columnar storage format, যা বড় ডেটাসেট সংরক্ষণের জন্য ব্যবহৃত হয়।

এটি .parquet এক্সটেনশন ব্যবহার করে।

Pandas-এ এটি পড়া যায় pd.read_parquet() ফাংশন দিয়ে।

```
1

2# parquet file

3parquet_file = pd.read_parquet('students.parquet')

4

5print(parquet_file)

6print(type(parquet_file))
```

আউটপুট:

```
1

2   StudentID          FullName  ...  Instructor    Location

30     PH1001       Alif Rahman  ...   Mr. Karim       Dhaka

41     PH1002     Fatima Akhter  ...   Ms. Salma  Chattogram

5...

619    PH1020        Nasir Khan  ...   Ms. Salma       Dhaka

7

8[20 rows x 9 columns]
```

```
<class 'pandas.core.frame.DataFrame'>
```

ব্যাখ্যা:

Parquet ফাইল সাধারণত Hadoop বা Spark এর মতো বিগ ডেটা সিস্টেমে ব্যবহৃত হয়।
এই ফরম্যাটে ডেটা কমপ্রেসড অবস্থায় থাকে, তাই এটি দ্রুত লোড হয় এবং মেমরি বাঁচায়।
এটি পড়তে হলে pyarrow বা fastparquet প্যাকেজ ইনস্টল থাকা প্রয়োজন।

4. JSON ফাইল লোড করা

JSON ফাইল মূলত key-value pair আকারে ডেটা রাখে।
এটি পড়তে pd.read_json() ফাংশন ব্যবহার করা হয়।

```
1

2# json file

3json_file = pd.read_json('data.json')

4

5print(json_file)

6print(type(json_file))
```

আউটপুট:

```
1

2  StudentID          FullName  ...  Algorithm Marks  Python Marks

30    PH1001       Alif Rahman  ...               85            88

41    PH1002     Fatima Akhter  ...               92            90

52    PH1004  Jannatul Ferdous  ...               78            82

6

7[3 rows x 5 columns]

8<class 'pandas.core.frame.DataFrame'>
```

ব্যাখ্যা:

JSON ফাইল সাধারণত nested বা structured data সংরক্ষণের জন্য ব্যবহৃত হয়।
pd.read_json() JSON কে DataFrame-এ রূপান্তর করে।
চাইলে DataFrame থেকে JSON ফাইল তৈরি করাও যায়:

```
1

2df.to_json('output.json')
```

Pandas Basic Functionalities

ভূমিকা

Pandas-এ DataFrame বা Series নিয়ে কাজ করার আগে এর কিছু মৌলিক (Basic) ফাংশন জানা খুব গুরুত্বপূর্ণ।
এই ফাংশনগুলোর মাধ্যমে আমরা ডেটাসেট সম্পর্কে দ্রুত ধারণা পেতে পারি — যেমন:

> কতগুলো রো বা কলাম আছে
> কোন কলামগুলোর নাম কী
> ডেটার ধরন (dtype)
> কোনো Missing Value আছে কি না
> কিছু সারির নমুনা দেখা ইত্যাদি

চলুন দেখি ধাপে ধাপে।

1. DataFrame লোড করা

প্রথমে student_data.csv ফাইলটি Pandas DataFrame আকারে লোড করা হবে।

```
1

2import pandas as pd

3

4df = pd.read_csv('student_data.csv')

5df
```

এখন df একটি DataFrame, যেখানে ছাত্রদের আইডি, নাম, ইনস্ট্রাক্টর, মার্কস ইত্যাদি তথ্য আছে।

2. প্রথম কিছু সারি দেখা (head)

head() ফাংশন DataFrame-এর প্রথম কয়েকটি সারি দেখায়।
ডিফল্টভাবে এটি প্রথম ৫টি সারি দেখায়, তবে চাইলে সংখ্যা নির্দিষ্ট করা যায়।

```
1

2# প্রথম ৪টি সারি দেখা

3df.head(4)
```

আউটপুট (সংক্ষেপে):

```
1
```

```
2  StudentID        FullName      Data Structure Marks  Algorithm Marks
30   PH1001       Alif Rahman                 85.0             82.0
41   PH1002    Fatima Akhter                  90.0             88.0
52   PH1003    Imran Hossain                  75.0             70.0
63   PH1004  Jannatul Ferdous                65.0             60.0
```

3. শেষের কিছু সারি দেখা (tail)

tail() ফাংশন DataFrame-এর শেষের কয়েকটি সারি দেখায়।

```
1
2# শেষের কয়েকটি সারি দেখা
3df.tail()
```

এটি সাধারণত ডেটাসেটের শেষ দিকের অবস্থা দেখতে ব্যবহৃত হয়, যেমন ইনপুট সম্পূর্ণ লোড হয়েছে কি না তা যাচাই করা।

4. কলাম সম্পর্কিত তথ্য দেখা

DataFrame-এর সব কলামের নাম দেখা যাবে columns অ্যাট্রিবিউট ব্যবহার করে।

```
1
2import numpy as np
3
4df.columns
5col = np.array(df.columns)
6print(col.dtype)
```

আউটপুট:

```
1
2object
```

ব্যাখ্যা:

> df.columns DataFrame-এর সব কলামের নাম একটি Index অবজেক্ট আকারে ফেরত দেয়।
> এটিকে NumPy array-এ রূপান্তর করলে দেখা যায় এর dtype হলো object, অর্থাৎ প্রতিটি কলাম নাম একটি স্ট্রিং।

5. ইনডেক্স দেখা

DataFrame-এর সারি নম্বর বা ইনডেক্স দেখতে df.index ব্যবহার করা হয়।

```
df.index
ind = np.array(df.index)
print(ind)
print(ind.dtype)
```

আউটপুট:

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19]
int64
```

ব্যাখ্যা:

- ডিফল্টভাবে Pandas প্রতিটি সারিকে 0, 1, 2, 3 … এইভাবে ইনডেক্স দেয়।
- ইনডেক্স সংখ্যা (integer type) হয়, তাই এর dtype হলো int64।

6. DataFrame সম্পর্কিত তথ্য দেখা (info)

info() ফাংশন ডেটাসেটের সারসংক্ষেপ দেয় — যেমন সারি সংখ্যা, কলাম সংখ্যা, প্রতিটি কলামের dtype, Missing Value ইত্যাদি।

```
df.info()
```

আউটপুট:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype 
---  ------                --------------  ----- 
0   StudentID             20 non-null     object
```

```
 1   FullName               20 non-null     object
 2   Data Structure Marks   16 non-null     float64
 3    Algorithm Marks        16 non-null     float64
 4   Python Marks           15 non-null     float64
 5   CompletionStatus       20 non-null     object
 6   EnrollmentDate         20 non-null     object
 7   Instructor             20 non-null     object
 8   Location               20 non-null     object
dtypes: float64(3), object(6)
memory usage: 1.5+ KB
```

ব্যাখ্যা:

- total 20টি রো ও 9টি কলাম আছে।
- কিছু কলামে missing value আছে (যেমন Python Marks)।
- ডেটার ধরন float64 ও object — অর্থাৎ সংখ্যাসূচক ও টেক্সট ডেটা আছে।

7. র‍্যান্ডম সারি দেখা (sample)

sample() ফাংশন DataFrame থেকে এলোমেলোভাবে কিছু সারি দেখায়।
এটি ডেটাসেটের একটা "random snapshot" দেয়।

```
df.sample(10)
```

এটি DataFrame থেকে এলোমেলোভাবে ১০টি সারি প্রদর্শন করবে।
প্রতিবার চালালে ভিন্ন সারি দেখা যাবে।

8. পরিসংখ্যানমূলক মান দেখা (describe)

describe() ফাংশন DataFrame-এর সংখ্যাসূচক কলামগুলোর গড়, মধ্যক, মানের পরিসীমা ইত্যাদি পরিসংখ্যান দেখায়।

```
df.describe()
```

আউটপুট (সংক্ষেপে):

```
2       Data Structure Marks  Algorithm Marks  Python Marks
3count              16.000000        16.000000     15.000000
4mean               77.500000        79.062500     81.200000
5std                10.235456         9.845670      7.325481
6min                60.000000        65.000000     70.000000
7max                95.000000        98.000000     92.000000
```

ব্যাখ্যা:

describe() ফাংশন সংখ্যাসূচক ডেটার উপর মৌলিক পরিসংখ্যান গণনা করে।

এটি Missing Value বাদ দিয়ে হিসাব করে।

Non-numeric কলামগুলোর ক্ষেত্রে আলাদা করে describe(include='object') ব্যবহার করা যায়।

Dataframe from Inbuilt Data with Column and Indexes

সব সময় আমাদের ডেটা কোনো CSV বা Excel ফাইলে থাকে না।

অনেক সময় ডেটা প্রোগ্রামের ভেতরেই থাকে — যেমন **list**, **tuple**, বা **dictionary** আকারে।

Pandas সহজেই এই ডেটাগুলো থেকে একটি **DataFrame** তৈরি করতে পারে।

1. লিস্ট (List) থেকে DataFrame তৈরি

List হলো একটি সাধারণ Python ডেটা স্ট্রাকচার, যেখানে একাধিক মান রাখা যায়।

একটি nested list (লিস্টের ভেতরে লিস্ট) থেকে DataFrame বানানো যায়।

```
1
2import pandas as pd
3
4my_list = [['Alice', 25], ['Bob', 30], ['Charlie', 28]]
5
6list_df = pd.DataFrame(my_list, columns=['Name', 'Age'], index=[1, 2, 3])
7
8print(type(list_df))
9list_df
```

আউটপুট:

```
<class 'pandas.core.frame.DataFrame'>
      Name  Age
0    Alice   25
1      Bob   30
2  Charlie   28
```

ব্যাখ্যা:

my_list হলো একটি লিস্টের ভেতরে লিস্ট।
columns দিয়ে কলামের নাম দেওয়া হয়েছে।
index দিয়ে সারির ইনডেক্স নির্ধারণ করা হয়েছে।
ফলাফল একটি পূর্ণাঙ্গ DataFrame।

2. টাপল (Tuple) থেকে DataFrame তৈরি

Tuple লিস্টের মতোই, তবে এর মান পরিবর্তন করা যায় না।

Tuple থেকেও DataFrame তৈরি করা সম্ভব।

```
my_tuple = (('Alice', 25), ('Bob', 30), ('Charlie', 28))

tuple_df = pd.DataFrame(my_tuple, columns=['Name', 'Age'])
tuple_df
```

আউটপুট:

```
      Name  Age
0    Alice   25
1      Bob   30
2  Charlie   28
```

ব্যাখ্যা:

এখানে ইনডেক্স না দিলে Pandas স্বয়ংক্রিয়ভাবে 0, 1, 2 ইনডেক্স সেট করে।

3. ডিকশনারি (Dictionary) থেকে DataFrame তৈরি

Dictionary হলো key-value জোড়ার ডেটা স্ট্রাকচার।

এটি DataFrame তৈরির সবচেয়ে প্রচলিত উপায়।

```
my_dict = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 29, 28]
}
```

```
6
7dict_df = pd.DataFrame(my_dict)
8dict_df
```

আউটপুট:

```
1
2      Name   Age
30    Alice   25
41      Bob   29
52  Charlie   28
```

ব্যাখ্যা:

Dictionary-এর key (যেমন 'Name', 'Age') হয় DataFrame-এর কলামের নাম।
আর এর value (যা লিস্ট আকারে থাকে) হয় সেই কলামের মান।

4. ডিকশনারির লিস্ট (List of Dictionaries) থেকে DataFrame তৈরি

একাধিক অবজেক্ট (যেমন ছাত্রদের তথ্য) আলাদা আলাদা dictionary-তে থাকলে
সেগুলো একটি লিস্টে রেখে DataFrame বানানো যায়।

```
1
2my_data = [
3    {'Name': 'Alice', 'Age': 25, 'City': 'New York'},
4    {'Name': 'Bob', 'Age': 30, 'City': 'Paris'},
5    {'Name': 'Charlie', 'Age': 28}
6]
7
8data_df = pd.DataFrame(my_data)
9data_df
```

আউটপুট:

```
1
2      Name   Age      City
30    Alice   25  New York
41      Bob   30     Paris
52  Charlie   28       NaN
```

ব্যাখ্যা:

প্রতিটি dictionary একটি করে রো হিসেবে যুক্ত হয়।
যেখানে কোনো মান অনুপস্থিত (যেমন Charlie-এর City), সেখানে Pandas স্বয়ংক্রিয়ভাবে NaN বসায়।
NaN মানে “Not a Number” বা Missing Value

5. DataFrame থেকে Series আলাদা করা

যখন কোনো নির্দিষ্ট কলাম নেওয়া হয়, সেটি Series আকারে পাওয়া যায়।

```
print(type(data_df['Name']))
```

আউটপুট:

```
<class 'pandas.core.series.Series'>
```

ব্যাখ্যা:

DataFrame হলো পুরো টেবিলের মতো।
Series হলো একক কলাম — একমাত্রিক ডেটা।

Accessing Data with Index and Column

DataFrame তৈরি করার পর সবচেয়ে গুরুত্বপূর্ণ কাজ হলো তার ভেতরের ডেটা দেখা ও নির্দিষ্ট অংশে অ্যাক্সেস করা।
Pandas-এ DataFrame থেকে ডেটা বের করার জন্য দুটি প্রধান উপায় রয়েছে —

কলামের নাম ব্যবহার করে
ইনডেক্স ও কলামের সমন্বয়ে **(loc/iloc)** ব্যবহার করে

1. পুরো **DataFrame** দেখা

```
df
```

এভাবে শুধু df লিখলে পুরো DataFrame টেবিল আকারে দেখা যায়।

2. একটি নির্দিষ্ট কলাম অ্যাক্সেস করা

একটি কলাম নেওয়ার জন্য কলামের নাম [] এর মধ্যে দিতে হয়।

```
# একটি কলাম অ্যাক্সেস করা
df['FullName']
```

আউটপুট:

```
0        Alice
1          Bob
2      Charlie
```

```
3         David
...
```

কলামের ডেটা সবসময় **Series** আকারে ফেরত দেয়।

```
type(df['FullName'])
```

আউটপুট:

```
pandas.core.series.Series
```

3. .loc[] ব্যবহার করে সারি (Row) অ্যাক্সেস করা

.loc[] হলো label-based accessor — অর্থাৎ, ইনডেক্স নাম বা লেবেল দিয়ে ডেটা পাওয়া যায়।

```
# নির্দিষ্ট একটি সারি অ্যাক্সেস
df.loc[0]
```

আউটপুট:

```
FullName              Alice
Python Marks             88
Algorithm Marks          90
CompletionStatus    Completed
Name: 0, dtype: object
```

একটি সারি অ্যাক্সেস করলে সেটিও **Series** আকারে ফেরত দেয়।

```
type(df.loc[0])
```

আউটপুট:

```
pandas.core.series.Series
```

4. একাধিক সারি অ্যাক্সেস করা

a.নির্দিষ্ট ইনডেক্স লিস্ট ব্যবহার করে

```
# একাধিক নির্দিষ্ট সারি
```

```
3df.loc[[2, 3, 19]]
```

আউটপুট:

```
1
2   FullName  Python Marks  Algorithm Marks CompletionStatus
32  Charlie          85.0               78         Completed
43    David          82.0               88       In Progress
519   Sarah          89.0               84         Completed
```

b. রেঞ্জ ব্যবহার করে

```
1
2# ইনডেক্স রেঞ্জ দিয়ে সারি নির্বাচন
3df.loc[3:7]
```

এইভাবে 3 থেকে 7 পর্যন্ত সারি পাওয়া যাবে (শেষ ইনডেক্সসহ)।

5. নির্দিষ্ট কলাম অ্যাক্সেস করা (loc ব্যবহার করে)

একক কলাম:

```
1
2df.loc[:, 'Python Marks']
```

আউটপুট:

```
1
20     88.0
31      NaN
42     85.0
53     82.0
6...
719    89.0
8Name: Python Marks, dtype: float64
```

একাধিক কলাম:

```
1
2df.loc[:, ['Python Marks', 'Algorithm Marks']]
```

আউটপুট:

```
1
2    Python Marks  Algorithm Marks
```

```
30          88.0                90
41           NaN                80
52          85.0                78
63          82.0                88
7...
8
9type(df.loc[:, ['Python Marks', 'Algorithm Marks']])
```

আউটপুট:

```
1
2pandas.core.frame.DataFrame
```

ব্যাখ্যা:
একাধিক কলাম নিলে ফলাফল DataFrame আকারে পাওয়া যায়।

6. নির্দিষ্ট সারি ও কলাম একসাথে নির্বাচন

.loc[] ব্যবহার করে নির্দিষ্ট সারি ও নির্দিষ্ট কলাম একসাথে সিলেক্ট করা যায়।

```
1
2df.loc[3:7, 'CompletionStatus']
```

আউটপুট:

```
1
23      Completed
34    In Progress
45      Completed
56    In Progress
67      Completed
7Name: CompletionStatus, dtype: object
```

ব্যাখ্যা:
এখানে ৩ থেকে ৭ নম্বর সারির জন্য শুধুমাত্র CompletionStatus কলাম নেওয়া হয়েছে।

Changing Index and Columns and iloc

DataFrame তৈরি করার পর অনেক সময় আমাদের প্রয়োজন হয় ইনডেক্স পরিবর্তন করা, কলামের নাম বদলানো, অথবা position অনুযায়ী ডেটা দেখা।
এই মডিউলে আমরা শিখব —

কিভাবে কোনো কলামকে নতুন ইনডেক্স হিসেবে ব্যবহার করা যায়
কিভাবে কলামের নাম পরিবর্তন করা যায়
এবং iloc ব্যবহার করে কিভাবে ডেটা দেখা যায়

1. কোনো কলামকে ইনডেক্স হিসেবে সেট করা

set_index() ফাংশনের মাধ্যমে আমরা DataFrame-এর ইনডেক্স পরিবর্তন করতে পারি।

```
1
2df_index = df.set_index('StudentID')
```

```
3df_index
```

এখানে 'StudentID' কলামটি এখন ইনডেক্স হিসেবে সেট হয়েছে।
অর্থাৎ, আগের সংখ্যাগত ইনডেক্স (0, 1, 2, …) এর জায়গায় এখন StudentID প্রদর্শিত হবে।

2. iloc দিয়ে ইনডেক্স নম্বর অনুসারে ডেটা দেখা

iloc হচ্ছে position-based indexing।
অর্থাৎ, এটি ইনডেক্সের নাম নয় — বরং অবস্থান (position) ধরে ডেটা অ্যাক্সেস করে।

```
1
2df_index.iloc[:, 0:5]
```

এই কোডের মানে —

: → সব সারি নেবে
0:5 → কলাম নম্বর 0 থেকে 4 পর্যন্ত (অর্থাৎ প্রথম পাঁচটি কলাম) নেবে

ফলাফল হিসেবে DataFrame-এর প্রথম পাঁচটি কলাম দেখা যাবে।

3. কলামের নাম পরিবর্তন করা

rename() ফাংশন ব্যবহার করে আমরা এক বা একাধিক কলামের নাম পরিবর্তন করতে পারি।
inplace=True দিলে মূল DataFrame-এই পরিবর্তন হবে।

```
1
2df.rename(columns={'FullName': 'Full Name', 'Algorithm Marks': 'Algo
Marks'}, inplace=True)
```

```
3df
```

এখানে —

'FullName' পরিবর্তিত হয়ে 'Full Name' হয়েছে
'Algorithm Marks' পরিবর্তিত হয়ে 'Algo Marks' হয়েছে

Dropping rows and columns,Assigning values and Iteration

ভূমিকা

DataFrame ব্যবহার করার সময় অনেক সময় প্রয়োজন হয় —

কোনো নির্দিষ্ট সারি (row) বা কলাম (column) মুছে ফেলা,
কোনো সেলে নতুন মান অ্যাসাইন করা,
অথবা সারি ধরে ডেটা ঘুরে দেখা (iteration করা)।

এই মডিউলে আমরা এই তিনটি কাজের ব্যবহার শিখব।

1. সারি (Row) মুছে ফেলা

drop() ফাংশনের মাধ্যমে আমরা ইনডেক্স অনুসারে সারি মুছে ফেলতে পারি।

```
1
2df.drop(0, inplace=True)
```

```
3df
```

এখানে 0 ইনডেক্সের সারিটি DataFrame থেকে বাদ যাবে।

inplace=True মানে হলো পরিবর্তনটি মূল df-এই প্রয়োগ হবে।

2. কলাম (Column) মুছে ফেলা

axis=1 দিয়ে বোঝানো হয় আমরা কলাম মুছতে চাই (axis=0 হলে সারি মুছে যায়)।

```
1
2df.drop('Instructor', axis=1, inplace=True)
```

```
3df
```

এখানে 'Instructor' কলামটি DataFrame থেকে সরানো হয়েছে।

3. নির্দিষ্ট মান পরিবর্তন বা অ্যাসাইন করা

iloc বা loc ব্যবহার করে কোনো নির্দিষ্ট সেল বা রেঞ্জে নতুন মান দেওয়া যায়।

```
1
2df.loc[1, 'Python Marks'] = 90
3df.loc[1, 'CompletionStatus'] = 'Completed'
```

```
4df.head(5)
```

এখানে ১ নম্বর সারির 'Python Marks' এবং 'CompletionStatus' আপডেট করা হয়েছে।

4. একাধিক মান একসাথে পরিবর্তন করা

একাধিক সারির নির্দিষ্ট কলামের মান একসাথে পরিবর্তন করা যায়।

```
df.loc[1:3, 'Python Marks'] += 2
df.head()
```

এখানে ১ থেকে ৩ নম্বর সারির 'Python Marks'-এর মান ২ করে বাড়ানো হয়েছে।

5. iterrows() দিয়ে iteration করা

iterrows() ব্যবহার করে DataFrame-কে row-by-row ঘুরে দেখা যায়।
প্রতিটি row একটি Series আকারে রিটার্ন হয়।

```
for i, series in df.iterrows():
    print(f"{i} : {series}")
```

এভাবে প্রতিটি সারির ইনডেক্স ও তার মান আলাদা করে দেখা যায়।

6. itertuples() দিয়ে দ্রুত iteration

itertuples() iterrows()-এর চেয়ে দ্রুত কাজ করে।
এটি প্রতিটি সারিকে namedtuple আকারে রিটার্ন করে।

```
for i in df.itertuples(index=False):
    print(i)
```

এখানে প্রতিটি সারির মান tuple আকারে দেখা যাবে, যা বেশি কার্যকরী ও দ্রুত।

Sorting a DF

sort_values() ফাংশন ব্যবহার করে DataFrame-কে নির্দিষ্ট কলাম বা একাধিক কলামের মান অনুযায়ী সাজানো যায়।
এটি ascending (ছোট থেকে বড়) বা descending (বড় থেকে ছোট) ক্রমে sort করতে পারে।

Example 1: DataFrame প্রদর্শন

```
1

2df
```

	StudentID	Full Name	Data Structure Marks	Algo Marks	Python Marks	CompletionStatus	EnrollmentDate	Location
1	PH1002	Fatima Akhter	92.0	92.0	92.0	Completed	2024-01-20	Chattogram
2	PH1003	Imran Hossain	88.0	88.0	91.0	Completed	2024-02-10	Dhaka
3	PH1004	Jannatul Ferdous	78.0	78.0	88.0	Completed	2024-02-12	Sylhet
4	PH1005	Kamal Uddin	90.0	NaN	95.0	In Progress	2024-03-05	Chattogram
5	PH1006	Laila Begum	75.0	75.0	78.0	Completed	2024-03-08	Rajshahi
6	PH1007	Mahmudul Hasan	80.0	80.0	NaN	In Progress	2024-04-01	Dhaka
7	PH1008	Nadia Islam	81.0	81.0	85.0	Completed	2024-04-22	Chattogram
8	PH1009	Omar Faruq	72.0	72.0	76.0	Completed	2024-05-16	Dhaka
9	PH1010	Priya Sharma	89.0	89.0	88.0	Completed	2024-05-20	Sylhet
10	PH1011	Rahim Sheikh	NaN	NaN	91.0	In Progress	2024-06-11	Khulna
11	PH1012	Sadia Chowdhury	85.0	85.0	87.0	Completed	2024-06-14	Chattogram
12	PH1013	Tanvir Ahmed	75.0	75.0	79.0	Completed	2024-07-02	Dhaka
13	PH1014	Urmi Akter	NaN	NaN	NaN	Not Started	2024-07-09	Rajshahi
14	PH1015	Wahiduzzaman	86.0	86.0	84.0	Completed	2024-08-18	Dhaka
15	PH1016	Ziaur Rahman	94.0	94.0	NaN	In Progress	2024-08-21	Chattogram
16	PH1017	Afsana Mimi	90.0	90.0	93.0	Completed	2025-09-01	Dhaka
17	PH1018	Babul Ahmed	88.0	88.0	85.0	Completed	2025-09-05	Sylhet
18	PH1019	Faria Rahman	NaN	NaN	NaN	Not Started	2025-09-15	Chattogram

Example 2: এক কলাম অনুযায়ী sort করা

```
1

2copy = df.sort_values('Data Structure Marks')
```

এখানে 'Data Structure Marks' কলামের মান অনুযায়ী ছোট থেকে বড় ক্রমে sort হয়েছে।

Example 3: Descending (বড় থেকে ছোট) ক্রমে sort করা

```
1

2copy = df.sort_values(['Data Structure Marks'], ascending=False)
```

এখানে DataFrame বড় থেকে ছোট মান অনুযায়ী সাজানো হয়েছে।

Example 4: একাধিক কলাম অনুযায়ী sort করা

```
1

2copy = df.sort_values(['Data Structure Marks', 'Python Marks'],
ascending=False)
```

প্রথমে 'Data Structure Marks' কলাম অনুযায়ী sort হবে।
যদি কোনো সারিতে ওই কলামের মান সমান হয়, তবে 'Python Marks' কলামের মান অনুযায়ী সাজানো হবে।

Example 5: একাধিক কলামে ভিন্ন ভিন্ন ক্রমে sort করা

```
1

2copy = df.sort_values(['Data Structure Marks', 'Python Marks'],
ascending=[0, 1])
```

এখানে 'Data Structure Marks' কলামটি descending (বড় থেকে ছোট)
এবং 'Python Marks' কলামটি ascending (ছোট থেকে বড়) ক্রমে সাজানো হয়েছে।

Filtering Data based on condition

DataFrame থেকে নির্দিষ্ট শর্ত অনুযায়ী সারি (rows) বেছে নেওয়া হয় **filtering** বা **conditional selection** দ্বারা।
এর জন্য সাধারণত loc[] ফাংশনের ভিতরে শর্ত (condition) ব্যবহার করা হয়।

Example 1: DataFrame প্রদর্শন

```
1

2df
```

	StudentID	Full Name	Data Structure Marks	Algo Marks	Python Marks	CompletionStatus	EnrollmentDate	Location
1	PH1002	Fatima Akhter	92.0	92.0	92.0	Completed	2024-01-20	Chattogram
2	PH1003	Imran Hossain	88.0	88.0	91.0	Completed	2024-02-10	Dhaka
3	PH1004	Jannatul Ferdous	78.0	78.0	88.0	Completed	2024-02-12	Sylhet
4	PH1005	Kamal Uddin	90.0	NaN	95.0	In Progress	2024-03-05	Chattogram
5	PH1006	Laila Begum	75.0	75.0	78.0	Completed	2024-03-08	Rajshahi
6	PH1007	Mahmudul Hasan	80.0	80.0	NaN	In Progress	2024-04-01	Dhaka
7	PH1008	Nadia Islam	81.0	81.0	85.0	Completed	2024-04-22	Chattogram
8	PH1009	Omar Faruq	72.0	72.0	76.0	Completed	2024-05-16	Dhaka
9	PH1010	Priya Sharma	89.0	89.0	88.0	Completed	2024-05-20	Sylhet
10	PH1011	Rahim Sheikh	NaN	NaN	91.0	In Progress	2024-06-11	Khulna
11	PH1012	Sadia Chowdhury	85.0	85.0	87.0	Completed	2024-06-14	Chattogram
12	PH1013	Tanvir Ahmed	75.0	75.0	79.0	Completed	2024-07-02	Dhaka
13	PH1014	Urmi Akter	NaN	NaN	NaN	Not Started	2024-07-09	Rajshahi
14	PH1015	Wahiduzzaman	86.0	86.0	84.0	Completed	2024-08-18	Dhaka
15	PH1016	Ziaur Rahman	94.0	94.0	NaN	In Progress	2024-08-21	Chattogram
16	PH1017	Afsana Mimi	90.0	90.0	93.0	Completed	2025-09-01	Dhaka
17	PH1018	Babul Ahmed	88.0	88.0	85.0	Completed	2025-09-05	Sylhet
18	PH1019	Faria Rahman	NaN	NaN	NaN	Not Started	2025-09-15	Chattogram

Example 2: নির্দিষ্ট শর্তে ডেটা ফিল্টার করা

```
1

2not_started = df.loc[df['CompletionStatus'] == 'Not Started']
```

এখানে 'CompletionStatus' কলামে যেসব সারিতে মান "Not Started" আছে,
শুধুমাত্র সেগুলোই not_started DataFrame-এ সংরক্ষিত হয়েছে।

Example 3: অন্য শর্তে ফিল্টার করা

```
1

2completed = df.loc[df['CompletionStatus'] == 'Completed']
```

এখানে 'CompletionStatus' কলামের মান "Completed" থাকা সব সারি দেখানো হয়েছে।

Example 4: একাধিক শর্ত একসাথে ব্যবহার করা

```
1

2completed_ds90 = df.loc[

3    (df['CompletionStatus'] == 'Completed') &

4    (df['Data Structure Marks'] >= 90)

5]
```

এই উদাহরণে দুটি শর্ত একসাথে প্রযোজ্য হয়েছে —

'CompletionStatus' হলো "Completed"

'Data Structure Marks' ≥ 90

দুটি শর্তই True হলে কেবল সেই সারিগুলোই ফিল্টার হয়ে completed_ds90 তে থাকবে।

Filtering Based on String

Pandas str.contains() মেথড

সাধারণ ব্যবহার

str.contains() মেথডটি Pandas DataFrame-এর কোনো কলামের স্ট্রিং ভ্যালুর মধ্যে নির্দিষ্ট প্যাটার্ন বা টেক্সট খুঁজে পেতে ব্যবহৃত হয়।

সিনট্যাক্স:

```
df[df['Column'].str.contains("pattern")]
```

✅ সাধারণ টেক্সট সার্চ (Case Sensitive)

```
df.loc[df['City'].str.contains("New")]
```

রেজাল্ট:

```
     Name       City Department  Salary
20    Alice   New York         HR   50000
32  Charlie     Newark    Finance   55000
44      Eve   New Delhi         HR   52000
56    Grace New Orleans    Finance   62000
```

ব্যাখ্যা:

City কলামে যেসব ভ্যালুর মধ্যে **"New"** টেক্সট আছে, শুধুমাত্র সেই রোগুলো সিলেক্ট করবে।

এটি **Case Sensitive**, অর্থাৎ বড় হাতের "N" এবং ছোট হাতের "ew" মিলতে হবে।

✅ Case Insensitive সার্চ

```
df.loc[df['City'].str.contains("new", case=False)]
```

রেজাল্ট: (উপরের মতোই)

```
     Name       City Department  Salary
20    Alice   New York         HR   50000
32  Charlie     Newark    Finance   55000
```

```
44      Eve    New Delhi         HR    52000

56    Grace New Orleans     Finance    62000
```

ব্যাখ্যা:

> case=False প্যারামিটার দেওয়ার কারণে এটি বড়-ছোট হাতের অক্ষর ধরবে না।
> "new", "New", "NEW" সবই ম্যাচ করবে।

✅ রেগুলার এক্সপ্রেশন (Regex) ব্যবহার

📑 শুরুতে কোনো টেক্সট মিলানো

```
1df.loc[df['City'].str.contains(r"^Los")]
```

রেজাল্ট:

```
1   Name         City Department  Salary
21   Bob  Los Angeles         IT   60000

38 Sakib    H Los Ang         HR   70000
```

ব্যাখ্যা:

> ^ চিহ্ন দিয়ে বুঝায় "শুরুতে মিলবে"।
> r"^Los" মানে সিটি নাম যেগুলো **Los** দিয়ে শুরু হয়।
> লক্ষ্য করুন: "H Los Ang" এও মিলেছে কারণ এতে "Los" আছে (শুরুতে না হলেও)।

📑সঠিকভাবে শুরুতে মিলানো জন্য:

```
1df.loc[df['City'].str.contains(r"^Los", case=False)]
```

📑 শেষে কোনো টেক্সট মিলানো

```
1df.loc[df['City'].str.contains(r"rk$")]
```

রেজাল্ট:

```
1      Name      City Department  Salary
20    Alice  New York         HR   50000

32  Charlie    Newark    Finance   55000
```

ব্যাখ্যা:

> $ চিহ্ন দিয়ে বুঝায় "শেষে মিলবে"।
> rk$ মানে সিটি নাম যেগুলো **rk** দিয়ে শেষ হয়।

📑 নাম যেগুলো **Vowel (A,E,I,O,U)** দিয়ে শুরু হয়

```
df.loc[df['Name'].str.contains(r"^[AEIOU]")]
```

রেজাল্ট:

```
   Name      City Department  Salary
0 Alice  New York         HR   50000
4   Eve  New Delhi         HR   52000
```

ব্যাখ্যা:

> ^[AEIOU] মানে নামটি যেকোনো Vowel (A,E,I,O,U) দিয়ে শুরু হতে হবে।
> এটি Case Sensitive, তাই শুধু বড় হাতের অক্ষর মিলবে।

📑 **Case Insensitive** ভাবে:

```
df.loc[df['Name'].str.contains(r"^[AEIOU]", case=False)]
```

📑 একাধিক প্যাটার্ন মিলানো (**OR** অপারেশন)

```
df.loc[df['City'].str.contains(r"New|Los")]
```

রেজাল্ট:

```
      Name         City Department  Salary
0    Alice     New York         HR   50000
1      Bob  Los Angeles         IT   60000
2  Charlie       Newark    Finance   55000
4      Eve     New Delhi         HR   52000
6    Grace   New Orleans    Finance   62000
8    Sakib    H Los Ang         HR   70000
```

ব্যাখ্যা:

> | চিহ্ন দিয়ে একাধিক প্যাটার্ন মিলানো যায় (OR অপারেশন)।
> এখানে "New" অথবা "Los" যেকোনোটি থাকলেই হবে।

✅রেগুলার এক্সপ্রেশন (Regex) এর গুরুত্বপূর্ণ চিহ্ন

চিহ্ন	অর্থ	উদাহরণ	ব্যাখ্যা
^	শুরু	^A	A দিয়ে শুরু হয়
$	শেষ	z$	z দিয়ে শেষ হয়
.	যেকোনো একটি অক্ষর	A.e	Ace, Abe, Aze etc.
*	০ বা তার বেশি বার	A*	'', A, AA, AAA
+	১ বা তার বেশি বার	A+	A, AA, AAA
[]	character set	[AEIOU]	যেকোনো একটি Vowel
|	|	OR অপারেশন	|A

Adding new Column and Saving the File

Pandas DataFrame-এ নতুন কলাম অ্যাড করা

Pandas DataFrame-এ নতুন কলাম অ্যাড করা একটি খুবই কমন এবং গুরুত্বপূর্ণ অপারেশন। এটি ডাটা অ্যানালাইসিস এবং ম্যানিপুলেশনের জন্য অত্যন্ত প্রয়োজনীয়।

✅ কনস্ট্যান্ট ভ্যালু সহ নতুন কলাম অ্যাড করা

```
1# 'Country' নামে নতুন কলাম অ্যাড করা যেখানে সবগুলোর ভ্যালু 'Bangladesh'

2df['Country'] = 'Bangladesh'
```

ব্যাখ্যা:

> df['Country'] - নতুন কলামের নাম
> = 'Bangladesh' - সব Row-এ একই ভ্যালু অ্যাসাইন করা হবে
> এটি Broadcasting নামে পরিচিত

✅ বিদ্যমান কলাম ব্যবহার করে নতুন কলাম তৈরি

```
1# তিনটি সাবজেক্টের মার্কস যোগ করে Total Marks কলাম তৈরি

2df['Total Marks'] = df['Data Structure Marks'] + df['Python Marks'] +
df['Algorithm Marks']
```

গুরুত্বপূর্ণ পয়েন্ট:

> যদি কোনো Row-এ NaN (Not a Number) থাকে, তাহলে Total-ও NaN হবে
> NaN মানে Missing Data

✅ শর্তভিত্তিক কলাম তৈরি

```
import numpy as np

# Data Structure Marks 90-এর বেশি হলে 'A+', নাহলে 'A'
df['A+ in DS'] = np.where(df['Data Structure Marks'] > 90, 'A+', 'A')
```

np.where() ফাংশন:

> Syntax: np.where(condition, value_if_true, value_if_false)
> এটি Pandas-এর ভেতর Conditional Logic ইমপ্লিমেন্ট করতে সাহায্য করে

✅ বুলিয়ান কলাম তৈরি

```
# Data Structure Marks 70-এর বেশি হলে True, নাহলে False
df['Passed in DS'] = df['Data Structure Marks'] > 70
```

বুলিয়ান ভ্যালু:

> True: শর্ত সত্য
> False: শর্ত মিথ্যা
> NaN থাকলে False হিসেবে বিবেচিত

✅ স্ট্রিং অপারেশন ব্যবহার করে কলাম তৈরি

```
# Full Name থেকে First Name আলাদা করা
df['First Name'] = df['FullName'].str.split(' ').str[0]
```

স্ট্রিং মেথডস:

> .str.split(' ') - স্পেস দিয়ে স্ট্রিংকে আলাদা করে List-এ পরিণত করে
> .str[0] - List-এর প্রথম এলিমেন্ট নেয়

Checking Duplicates and Null Values in DF

Pandas-এ ডেটা অ্যানালাইসিসের সময় আমাদের প্রায়ই বিভিন্ন ভ্যালুর ইউনিক ভ্যালু বের করা, null ভ্যালু চেক করা ইত্যাদি কাজ করতে হয়। আজ আমরা শিখব:

> unique() এবং nunique() মেথড
> isnull() এবং notnull() মেথড
> hasnans প্রপার্টি

✅ unique() এবং nunique() মেথড

📑 unique() মেথড

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie", "Alice", "David", "Bob"],
    "City": ["New York", "London", "Paris", "New York", "Tokyo", "London"],
    "Score": [85, 90, 78, 85, 95, 90]
}

df = pd.DataFrame(data)
print(df)
```

আউটপুট:

```
      Name      City  Score
0    Alice  New York     85
1      Bob    London     90
2  Charlie     Paris     78
3    Alice  New York     85
4    David     Tokyo     95
5      Bob    London     90
```

📑 unique() শুধুমাত্র Series-এ কাজ করে

```
# Name কলামের ইউনিক ভ্যালু
unique_names = df['Name'].unique()
print(unique_names)
```

আউটপুট:

```
array(['Alice', 'Bob', 'Charlie', 'David'], dtype=object)
```

গুরুত্বপূর্ণ নোট:

> unique() শুধুমাত্র একটি **Series-**এ কাজ করে
> সম্পূর্ণ DataFrame-এ সরাসরি unique() ব্যবহার করা যায় না

এটি একটি **numpy array** রিটার্ন করে

📑 nunique() মেথড

```
# Name কলামে কতগুলো ইউনিক ভ্যালু আছে
nunique_names = df['Name'].nunique()
print(f"ইউনিক নামের সংখ্যা: {nunique_names}")
```

আউটপুট:

```
ইউনিক নামের সংখ্যা: 4
```

✅DataFrame-এ nununique() ব্যবহার

```
# প্রতিটি কলামের ইউনিক ভ্যালুর সংখ্যা
print(df.nunique())
```

আউটপুট:

```
Name     4
City     4
Score    4
dtype: int64
```

📑 Data Structure Marks কলামের ইউনিক ভ্যালু

```
# ইউনিক ভ্যালুর সংখ্যা
unique_count = len(df1['Data Structure Marks'].unique())
print(f"ইউনিক মার্কসের সংখ্যা: {unique_count}")
```

```
4
```

```
# অথবা সরাসরি nunique() ব্যবহার
nunique_count = df1['Data Structure Marks'].nunique()
print(f"nunique() দিয়ে গণনা: {nunique_count}")
```

আউটপুট:

```
ইউনিক মার্কসের সংখ্যা: 12
```

```
nunique() দিয়ে গণনা: 12
```

✅ Null Value চেকিং

📑 isnull() মেথড

```
# সম্পূর্ণ DataFrame-এ null ভ্যালু চেক
null_check = df1.isnull()

print(null_check)
```

ব্যাখ্যা:

> True মানে সেই সেলটি null
> False মানে সেই সেলটি null নয়

📑 নির্দিষ্ট কলামে null চেক

```
# শুধুমাত্র Data Structure Marks কলামে null চেক
ds_marks_null = df1['Data Structure Marks'].isnull()

print(ds_marks_null)
```

আউটপুট:

```
0     False
1     False
2     False
3     False
4      True
5     False
6     False
7     False
8     False
9     False
10     True
11    False
12    False
13     True
14    False
15    False
```

```
16    False
17    False
18     True
19    False
Name: Data Structure Marks, dtype: bool
```

📑 notnull() মেথড

```
# null নয় এমন ভ্যালু চেক
ds_marks_notnull = df1['Data Structure Marks'].notnull()
print(ds_marks_notnull)
```

ব্যাখ্যা:

> True মানে সেই সেলটি null নয়
> False মানে সেই সেলটি null

✅ hasnans প্রপার্টি

```
# কলামে কোনো null ভ্যালু আছে কিনা চেক
has_nulls = df1['Data Structure Marks'].hasnans
print(f"Data Structure Marks-এ null আছে: {has_nulls}")

# StudentID কলাম চেক
student_id_nulls = df1['StudentID'].hasnans
print(f"StudentID-তে null আছে: {student_id_nulls}")
```

আউটপুট:

```
Data Structure Marks-এ null আছে: True
StudentID-তে null আছে: False
```

📑 শুধুমাত্র null ভ্যালুওয়ালা রো দেখানো

```
# যেসব স্টুডেন্টের Data Structure Marks null
null_ds_marks = df1[df1['Data Structure Marks'].isnull()]

print(null_ds_marks[['FullName', 'StudentID', 'Data Structure Marks']])
```

📑 সম্পূর্ণ ডেটা দেখানো

```
1# শুধুমাত্র নাম এবং ID দেখানো
2selected_columns = df1[['FullName', 'StudentID']]
3print(selected_columns)
```

📑 ইউনিক ভ্যালু based ফিল্টারিং

```
1# শুধুমাত্র যেসব শহরের স্টুডেন্ট আছে
2unique_cities = df1['Location'].unique()
3print(f"সকল শহর: {unique_cities}")
```

unique() → শুধুমাত্র Series-এ, array return করে
nunique() → Series/DataFrame উভয়য়তে, count return করে
isnull() → null ভ্যালু খুঁজে, boolean mask return করে
notnull() → null নয় এমন ভ্যালু খুঁজে
hasnans → দ্রুত null আছে কিনা চেক করার জন্য

মেথড	কাজ	রিটার্ন টাইপ
unique()	ইউনিক ভ্যালু বের করে	numpy array
nunique()	ইউনিক ভ্যালুর সংখ্যা	integer
isnull()	null ভ্যালু চেক	boolean Series
notnull()	non-null ভ্যালু চেক	boolean Series
hasnans	null আছে কিনা	boolean

Handling Duplicates Values

Pandas-এ Duplicate Values হ্যান্ডলিং

ডাটাফ্রেমে যখন একই রকম ডাটা একাধিকবার থাকে, তখন তাকে ডুপ্লিকেট ভ্যালু বলে।
যেমন: একই নাম, একই শহর বা একই স্কোর একাধিকবার থাকতে পারে।

✅ ডুপ্লিকেট চেক করা

📑 duplicated() মেথড

> এটি প্রতিটি row-এর জন্য True বা False রিটার্ন করে।
> True মানে সেই rowটি ডুপ্লিকেট।

উদাহরণ:

```
1df.duplicated()
```

আউটপুট:

```
10    False
21    False
32    False
43    False
54    False
65     True
7dtype: bool
```

📑 মোট ডুপ্লিকেট সংখ্যা বের করা

```
1df.duplicated().sum()
```

আউটপুট: `1`

✅ ডুপ্লিকেট রিমুভ করা

📑 drop_duplicates() মেথড

> ডিফল্টভাবে এটি প্রথম occurrence রাখে এবং পরের ডুপ্লিকেটগুলো রিমুভ করে দেয়।

উদাহরণ:

```
1df.drop_duplicates()
```

আউটপুট:

```
1      Name      City  Score
20    Alice  New York     85
31      Bob    London     90
42  Charlie     Paris     78
53    Alice  New York     70
64    David     Tokyo     95
```

📑 পরমানেন্টভাবে রিমুভ করা

```
1df.drop_duplicates(inplace=True)
```

> inplace=True ব্যবহার করলে আসল ডাটাফ্রেমই পরিবর্তন হয়ে যায়।

✅ নির্দিষ্ট কলাম অনুযায়ী ডুপ্লিকেট রিমুভ করা

📑 **subset** প্যারামিটার

> শুধুমাত্র নির্দিষ্ট কলামের ভিত্তিতে ডুপ্লিকেট চেক করা যায়।

নামের ভিত্তিতে ডুপ্লিকেট রিমুভ করা:

```
1df.drop_duplicates(subset=['Name'])
```

আউটপুট:

```
1       Name      City  Score
20     Alice  New York     85
31       Bob    London     90
42   Charlie     Paris     78
54     David     Tokyo     95
```

শহরের ভিত্তিতে ডুপ্লিকেট মুছুন:

```
1df.drop_duplicates(subset=['City'])
```

আউটপুট:

```
1       Name      City  Score
20     Alice  New York     85
31       Bob    London     90
42   Charlie     Paris     78
54     David     Tokyo     95
```

✅ডুপ্লিকেট এর মধ্যে কোন গুলো রাখবেন?

keep প্যারামিটার

> keep='first' → প্রথম occurrence রাখে (ডিফল্ট)
> keep='last' → শেষ occurrence রাখে
> keep=False → কোনো ডুপ্লিকেটই রাখে না

শেষ **occurrence** রাখুন:

```
df.drop_duplicates(subset=['Name'], keep='last')
```

আউটপুট:

```
      Name      City  Score
1      Bob    London     90
2  Charlie     Paris     78
3    Alice  New York     70
4    David     Tokyo     95
```

✅কোনো ডুপ্লিকেটই রাখবেন না:

```
df.drop_duplicates(subset=['Name'], keep=False)
```

> এখানে Alice এবং Bob দুজনই সম্পূর্ণরূপে মুছে যাবে।

Handling Null Values

✅ Pandas-এ Null ভ্যালু হ্যান্ডলিং

📑 Null ভ্যালু কি?

> ডাটাসেটে যখন কোনো ডাটা মিসিং বা অনুপস্থিত থাকে, তখন সেটিকে Null ভ্যালু বা NaN (Not a Number) বলে।
> Pandas-এ NaN হিসেবে দেখানো হয়।

✅Null ভ্যালু ডিলেট করার উপায়

📑 dropna() মেথড

> সব রো যেখানে যেকোনো কলামে Null ভ্যালু আছে, সেগুলো ডিলেট করে।

```
df.dropna()
```

📑 `how='all'` প্যারামিটার

> শুধুমাত্র সেই রো গুলো ডিলেট করে যেখানে সব কলামে Null ভ্যালু আছে।

```
df.dropna(how='all', inplace=True)
```

> inplace=True দিলে আসল ডাটাফ্রেমে পরিবর্তন সেভ হয়।

📑 **subset** প্যারামিটার

শুধুমাত্র নির্দিষ্ট কলামে Null ভ্যালু থাকলে সেই রো ডিলেট করে।

```
# শুধুমাত্র 'Python Marks' কলামে Null থাকলে রো ডিলেট করবে
df.dropna(subset=['Python Marks'])

# একাধিক কলামের জন্য
df.dropna(subset=['Python Marks', 'Algorithm Marks'])
```

✅ Null ভ্যালু রিপ্লেস করার উপায়

📑 **fillna()** মেথড

সব Null ভ্যালুকে একটি নির্দিষ্ট ভ্যালু দিয়ে রিপ্লেস করে।

```
df.fillna(0)
```

📑 নির্দিষ্ট কলামের **Null** ভ্যালু রিপ্লেস করা

```
# 'FullName' কলামের Null ভ্যালুকে 'unknown' দিয়ে রিপ্লেস করা
df['FullName'].fillna('unknown')
```

📑 **Mean/Median/Mode** দিয়ে রিপ্লেস করা

নিউমেরিক কলামের Null ভ্যালুকে Mean/Median/Mode দিয়ে রিপ্লেস করা যায়।

```
# 'Python Marks' কলামের Null ভ্যালুকে Mean দিয়ে রিপ্লেস করা
df['Python Marks'].fillna(df['Python Marks'].mean(), inplace=True)
```

inplace=True দিলে আসল ডাটাফ্রেমে পরিবর্তন সেভ হয়।

✅গুরুত্বপূর্ণ পয়েন্টস

dropna() vs fillna():

dropna(): Null ভ্যালু সরিয়ে দেয় (রো বা কলাম ডিলেট করে)।

fillna(): Null ভ্যালুকে নতুন ভ্যালু দিয়ে রিপ্লেস করে।

inplace=True:

ডাটাফ্রেমে পরিবর্তন সেভ করতে চাইলে ব্যবহার করুন।

inplace=False (ডিফল্ট) দিলে নতুন ডাটাফ্রেম রিটার্ন করে, আসল ডাটাফ্রেম অপরিবর্তিত থাকে।

subset ব্যবহার:

শুধুমাত্র প্রয়োজনীয় কলামে Null ভ্যালু চেক করার জন্য ব্যবহার করুন।

Mean/Median/Mode ব্যবহার:

নিউমেরিক ডাটার জন্য উপযুক্ত।

Categorical ডাটার জন্য Mode ব্যবহার করা ভালো।

✅ প্র্যাকটিকাল উদাহরণ

```
# ডাটাসেট লোড
df = pd.read_csv('student_data.csv')

# শুধুমাত্র সম্পূর্ণ Null রো ডিলেট
df.dropna(how='all', inplace=True)

# 'Python Marks' এর Null ভ্যালু Mean দিয়ে ফিল
df['Python Marks'].fillna(df['Python Marks'].mean(), inplace=True)

# 'FullName' এর Null ভ্যালু 'Unknown' দিয়ে ফিল
df['FullName'].fillna('Unknown', inplace=True)
```

✅ কোন পদ্ধতি কখন ব্যবহার করবেন?

যদি **Null** ভ্যালু কম থাকে → dropna() ব্যবহার করুন।

যদি **Null** ভ্যালু বেশি থাকে → fillna() ব্যবহার করুন।

নিউমেরিক ডাটা হলে → Mean/Median দিয়ে ফিল করুন।

ক্যাটেগরিক্যাল ডাটা হলে → Mode বা নতুন ক্যাটেগরি দিয়ে ফিল করুন।

Statistical Functions in Pandas

Pandas Statistical Functions

✅ ডেটাফ্রেম প্রিপারেশন

আমরা একটি স্টুডেন্ট ডেটাসেট ব্যবহার করবো যেখানে বিভিন্ন বিষয়ের মার্কস, কমপ্লিশন স্ট্যাটাস, ইনস্ট্রাক্টর, লোকেশন ইত্যাদি তথ্য আছে।

```
import pandas as pd
import numpy as np

# ডেটাফ্রেম লোড
df = pd.DataFrame({
    'StudentID': ['PH1001', 'PH1002', ..., 'PH1020', None],
    'FullName': ['Alif Rahman', 'Fatima Akhter', ..., None],
    # ... অন্যান্য কলাম
})
```

📑 dropna() - মিসিং ভ্যালু মুছে ফেলা

যে কোনো row-এ যদি একটি কলামেও NaN (Not a Number) থাকে, তাহলে পুরো row টি ডিলিট করে দেবে।

```
cleaned_df = df.dropna()
print(cleaned_df)
```

ব্যাখ্যা:

> dropna() ডিফল্টভাবে যেকোনো row যেখানে একটি কলামেও **NaN** থাকে, সেটিকে মুছে দেয়।
> আমাদের ডেটাসেটে ২১টি row ছিল, dropna() প্রয়োগের পর ১৬টি row ।

📑 sum() - যোগফল বের করা

একটি কলামের সকল সংখ্যার যোগফল বের করে।

```
total_marks = df['Data Structure Marks'].sum()
print(total_marks)  # আউটপুট: 1344.0
```

ব্যাখ্যা:

> শুধুমাত্র numeric ডেটা থাকলে কাজ করবে।
> NaN values উপেক্ষা করে যোগফল বের করে।

📑 min() ও max() - সর্বনিম্ন ও সর্বোচ্চ মান

```
min_mark = df['Data Structure Marks'].min()
```

```
2max_mark = df['Data Structure Marks'].max()
3print(min_mark)  # 72.0
4print(max_mark)  # 94.0
```

📑 mean() - গড় মান

কলামের সকল সংখ্যার গড় বের করে।

```
1average_mark = df['Data Structure Marks'].mean()
2print(average_mark)  # 84.0
```

📑 median() - মধ্যমান

ডেটাকে সাজানোর পর মাঝের মান।

```
1median_mark = df['Data Structure Marks'].median()
2print(median_mark)  # 85.5
```

📑 mode() - সর্বাধিক-occurring-value

যে মানটি সবচেয়ে বেশি বার repeated হয়েছে।

```
1mode_mark = df['Data Structure Marks'].mode()
2print(mode_mark)
3# Output:
4# 0    75.0
5# 1    85.0
6# 2    86.0
7# 3    88.0
```

নোট: একাধিক মান থাকলে সবগুলো return করে।

📑 std() - স্ট্যান্ডার্ড ডেভিয়েশন

ডেটার "ছড়ানো" বা variability measure করে।

```
1std_dev = df['Data Structure Marks'].std()
2print(std_dev)  # 6.501281924871945
```

ব্যাখ্যা:

> STD যত কম, ডেটা তত বেশি কাছাকাছি।
> STD যত বেশি, ডেটা তত বেশি ছড়ানো।

📑 corr() - কোরিলেশন ম্যাট্রিক্স

দুইটি numerical কলামের সম্পর্ক measure করে।

```
correlation_matrix = df[['Data Structure Marks', 'Python Marks']].corr()
print(correlation_matrix)
```

আউটপুট:

```
                      Data Structure Marks  Python Marks
Data Structure Marks              1.000000      0.776845
Python Marks                      0.776845      1.000000
```

ব্যাখ্যা:

> 1.000000 → নিজের সাথে পারফেক্ট সম্পর্ক।
> 0.776845 → Strong positive correlation (একটি বাড়লে অন্যটিও বাড়ে)।

📑 Row-wise Sum (axis=1)

একাধিক কলামের ভ্যালু row-wise যোগ করতে:

```
rowwise_sum = df[['Data Structure Marks', 'Python Marks']].sum(axis=1)
print(rowwise_sum)
```

নোট: axis=1 মানে row-wise অপারেশন।

📑 নতুন কলাম যোগ করা

উপরের টেকনিক ব্যবহার করে নতুন কলাম বানানো:

```
df['Total Marks'] = df.iloc[:, 2:5].sum(axis=1)
```

ব্যাখ্যা:

> iloc[:, 2:5] → ২য়, ৩য়, ৪র্থ কলাম (Python-এ start inclusive, end exclusive)।
> sum(axis=1) → প্রতিটি row-এর জন্য আলাদাভাবে যোগ করবে।

📑 describe() - সম্পূর্ণ Statistical Summary

সব numerical কলামের একসাথে statistical summary দেয়।

```
summary = df.describe()
print(summary)
```

আউটপুটে যা পাবেন:

- count → নন-নাল ভ্যালুর সংখ্যা
- mean → গড়
- std → স্ট্যান্ডার্ড ডেভিয়েশন
- min → সর্বনিম্ন
- 25% → প্রথম কোয়ার্টাইল
- 50% → মধ্যমান (median)
- 75% → তৃতীয় কোয়ার্টাইল
- max → সর্বোচ্চ

Apply Function in DF

Pandas Apply() ফাংশন

✅ apply() ফাংশন কি?

apply() হলো pandas এর একটি শক্তিশালী ফাংশন যা DataFrame বা Series এর উপর custom ফাংশন প্রয়োগ করতে ব্যবহৃত হয়।

✅ কেন apply() ব্যবহার করব?

- custom লজিক প্রয়োগ করতে
- একই অপারেশন multiple columns/rows এ প্রয়োগ করতে
- DataFrame এর ডেটা প্রসেসিং সহজ করতে

✅ Series এর উপর apply() ব্যবহার

📑 Lambda ফাংশন ব্যবহার

```
# Min-Max Scaling
mn = df['Total Marks'].min()
mx = df['Total Marks'].max()

df['Scaled Marks'] = df['Total Marks'].apply(lambda x : (x-mn)/(mx-mn))
```

ব্যাখ্যা:

- lambda x : (x-mn)/(mx-mn) - একটি সরল ফাংশন যেটি প্রতিটি মার্ককে ০-১ এর মধ্যে স্কেল করে
- apply() প্রতিটি row এর 'Total Marks' কলামে এই ফাংশন প্রয়োগ করে

📑 Custom ফাংশন ব্যবহার

```
def grading_system(marks):
    if marks >= 260:
        return 'A+'
    elif marks >= 250:
        return 'A'
    else:
        return 'A-'

df['Grade'] = df['Total Marks'].apply(grading_system)
```

ব্যাখ্যা:

> grading_system() একটি custom ফাংশন যা মার্কস অনুযায়ী গ্রেড দেয়
> apply() প্রতিটি student এর মার্কস নিয়ে এই ফাংশনে পাঠায়

✅ DataFrame এর উপর apply() ব্যবহার

📑 Multiple Columns ব্যবহার

```
def marking_system(df):
    a = df['Data Structure Marks'] * 2
    b = df['Python Marks'] * 3
    c = df['Algorithm Marks'] * 4
    return a + b + c

df['Exceptional Marks'] = df.apply(marking_system, axis=1)
```

গুরুত্বপূর্ণ পয়েন্ট:

> axis=1 দিলে ফাংশনটি **row-wise** কাজ করে (প্রতিটি row আলাদাভাবে প্রসেস হয়)
> axis=0 দিলে ফাংশনটি **column-wise** কাজ করে
> একসাথে multiple columns এক্সেস করতে পারি

✅ apply() এর সুবিধা

📑 কোডের পুনর্ব্যবহার

একই ফাংশন বিভিন্ন জায়গায় ব্যবহার করতে পারি

📑 পড়তে সহজ

```
# Without apply - Complex
df['Grade'] = ['A+' if x>=260 else 'A' if x>=250 else 'A-' for x in df['Total Marks']]

# With apply - Clean
df['Grade'] = df['Total Marks'].apply(grading_system)
```

📑 জটিল লজিক সহজে প্রয়োগ
Complex calculations সহজে handle করতে পারি

✅ ব্যবহারের নিয়ম

📑 **Series** এর জন্য

```
# Single column
df['New_Col'] = df['Existing_Col'].apply(function_name)
```

📑 **DataFrame** এর জন্য

```
# Multiple columns
df['New_Col'] = df.apply(function_name, axis=1)
```

📑 **Lambda vs Custom Function**

```
# Simple operations - Lambda
df['Double_Marks'] = df['Total Marks'].apply(lambda x: x*2)

# Complex operations - Custom Function
def complex_calculation(x):
    # multiple steps
    # conditions
    return result

df['New_Col'] = df['Col'].apply(complex_calculation)
```

DateTime and Timedelta

✅ ডেটাটাইম (DateTime) কি?

ডেটাটাইম হল একটি বিশেষ ডেটা টাইপ যা তারিখ এবং সময় একসাথে সংরক্ষণ করে। Pandas-এ datetime64[ns] টাইপ ব্যবহার করে আমরা তারিখ-সময় সম্পর্কিত ডেটা সহজে ম্যানিপুলেট করতে পারি।

✅ প্রধান ধাপসমূহ:

```
import pandas as pd

df = pd.read_csv('student_completed_data.csv')
```

📑 তারিখের কলামকে ডেটাটাইমে কনভার্ট করা

আমাদের ডেটাসেটে EnrollmentDate এবং FinishedDate কলামগুলো অবজেক্ট/স্ট্রিং টাইপে আছে। এগুলোকে ডেটাটাইমে কনভার্ট করতে হবে:

```
df['EnrollmentDate'] = pd.to_datetime(df['EnrollmentDate'])
df['FinishedDate'] = pd.to_datetime(df['FinishedDate'])
```

লক্ষণীয়:

pd.to_datetime() ফাংশনটি স্ট্রিং/অবজেক্ট ডেটাকে Pandas-এর ডেটাটাইম ফরমেটে কনভার্ট করে।

📑 ডেটাটাইম প্রোপার্টি ব্যবহার করে নতুন কলাম তৈরি

ডেটাটাইমে কনভার্ট করার পর আমরা বিভিন্ন প্রোপার্টি এক্সেস করতে পারি:

```
df['Enrollment Year'] = df['EnrollmentDate'].dt.year
df['Enrollment Date'] = df['EnrollmentDate'].dt.day
```

ব্যবহৃত প্রোপার্টি:

> .dt.year → বছর বের করে (2024)
> .dt.day → মাসের কত তারিখ (1-31)
> .dt.month → মাস (1-12)
> .dt.day_name() → বার (Monday, Tuesday...)

📑 টাইমডেল্টা (TimeDelta) হিসাব

দুইটি তারিখের মধ্যে পার্থক্য বের করতে টাইমডেল্টা ব্যবহার করা হয়:

```
df['Total time taken to finish'] = df['FinishedDate'] -
df['EnrollmentDate']
```

এখানে FinishedDate থেকে EnrollmentDate বিয়োগ করলে আমরা পাচ্ছি কতদিন সময় লাগেছে কোর্স শেষ করতে।
আউটপুট ফরমেট:
113 days, 88 days ইত্যাদি

✅ গুরুত্বপূর্ণ মেথড ও প্রোপার্টিসমূহ:

📑 ডেটাটাইম কনভার্শন:

pd.to_datetime(column) → স্ট্রিং/অবজেক্টকে ডেটাটাইমে কনভার্ট করে

📑 ডেটাটাইম এক্সট্রাকশন:

.dt.year → বছর
.dt.month → মাস
.dt.day → তারিখ
.dt.hour → ঘন্টা
.dt.minute → মিনিট
.dt.second → সেকেন্ড
.dt.day_name() → বারের নাম
.dt.month_name() → মাসের নাম

📑 টাইমডেল্টা অপারেশন:

date1 - date2 → দুটি তারিখের পার্থক্য
.dt.days → শুধু দিন সংখ্যা পেতে
.dt.total_seconds() → মোট সেকেন্ডে পেতে

✅ টাইমডেল্টা থেকে সংখ্যায় কনভার্ট:

```
1# শুধু দিন সংখ্যা পেতে
2df['Days_taken'] = df['Total time taken to finish'].dt.days
3
4# সপ্তাহে কনভার্ট করতে
5df['Weeks_taken'] = df['Days_taken'] / 7
```

✅ সতর্কতা:

তারিখ কনভার্ট করার আগে ডেটা ক্লিন আছে কিনা নিশ্চিত হোন
তারিখের ফরমেট সঠিক কিনা চেক করুন (YYYY-MM-DD)
Null/NaN ভ্যালু থাকলে সেগুলো হ্যান্ডেল করুন

✅ ব্যবহারের সুবিধা:

সময়সীমা ভিত্তিক ডেটা অ্যানালাইসিস
বিভিন্ন সময়কালের তুলনা
ট্রেন্ড অ্যানালাইসিস
রিপোর্টিং এবং ভিজুয়ালাইজেশন

Groupby in DF

Pandas GroupBy এবং DataFrame Operations

✅ **DataFrame** এর বেসিক ইনফরমেশন

```
# DataFrame দেখানো
df

# DataFrame এর আকার (rows, columns)
df.shape

# কলামগুলোর নাম
df.columns

# ডেটা টাইপস
df.dtypes
```

✅ **Column** ড্রপ করা (অপ্রয়োজনীয় কলাম সরানো)

```
# StudentID, FullName, CompletionStatus কলাম ডিলিট করা
```

```
2df.drop(columns=['StudentID','FullName','CompletionStatus'],
inplace=True)
```

ব্যাখ্যা:

> drop(): কলাম বা row ডিলিট করতে ব্যবহৃত হয়
> columns=['col1','col2']: কোন কলামগুলো ড্রপ করবে
> inplace=True: মূল DataFrame এ পরিবর্তন সেভ করবে (False করলে নতুন DataFrame রিটার্ন করবে)

✅ GroupBy অপারেশন

📑 গ্রুপ তৈরি করা

```
1group = df.groupby('Instructor')
```

GroupBy কী?

> ডেটাকে বিভিন্ন গ্রুপে ভাগ করে
> প্রতিটি গ্রুপের জন্য আলাদা আলাদা অপারেশন চালানো যায়
> SQL এর GROUP BY এর মতোই কাজ করে

✅ বিভিন্ন **Aggregation** ফাংশন

📑 **sum() -** যোগফল

```
1group.sum()
```

আউটপুট বিশ্লেষণ:

> Mr. David: 2399 মার্কস, 662 দিন
> Mr. Karim: 1911 মার্কস, 658 দিন
> Ms. Salma: 1049 মার্কস, 335 দিন

📑 **min() -** সর্বনিম্ন মান

```
1group.min()
```

আউটপুট বিশ্লেষণ:

> Mr. David: সর্বনিম্ন 236 মার্কস, 43 দিন
> Mr. Karim: সর্বনিম্ন 253 মার্কস, 80 দিন
> Ms. Salma: সর্বনিম্ন 234 মার্কস, 40 দিন

📑 **max() -** সর্বোচ্চ মান

```
1group.max()
```

আউটপুট বিশ্লেষণ:

Mr. David: সর্বোচ্চ 291 মার্কস, 115 দিন
Mr. Karim: সর্বোচ্চ 291 মার্কস, 120 দিন
Ms. Salma: সর্বোচ্চ 300 মার্কস, 113 দিন

📑 first() - প্রতিটি গ্রুপের প্রথম row

```
group.first()
```

📑 last() - প্রতিটি গ্রুপের শেষ row

```
group.last()
```

✅ Conditional Selection (শর্তসাপেক্ষ সিলেকশন)

📑 Boolean Mask তৈরি করা

```
# যেসব students এর মার্কস 250 এর বেশি
mask = df['Total Marks'] > 250
print(mask)
```

📑 Boolean Mask ব্যবহার করে কাউন্ট করা

```
# কতজন student এর মার্কস 250 এর বেশি
count_above_250 = (df['Total Marks'] > 250).sum()
print(f"250+ মার্কস প্রাপ্ত students: {count_above_250} জন")
```

✅ Advanced GroupBy Operations

📑 সর্টিং এর সাথে GroupBy

```
# মার্কস অনুসারে ডিসেন্ডিং অর্ডারে সর্ট করে, প্রতিটি ইনস্ট্রাক্টরের top 5 students


top_students = df.sort_values('Total Marks',
ascending=False).groupby('Instructor').head(5)
```

ব্যাখ্যা:

sort_values('Total Marks', ascending=False): মার্কস অনুসারে ডিসেন্ডিং অর্ডারে সর্ট
groupby('Instructor'): ইনস্ট্রাক্টর অনুসারে গ্রুপ
head(5): প্রতিটি গ্রুপের প্রথম ৫টি row নেয়া